Le format DIMACS pour SAT : Dans ce format très simple, les variables sont des entiers (> 0). La négation d'une variable se fait grâce au signe de l'entier (-1 correspond au littéral NON 1). Chaque ligne du fichier correspond à une clause de la formule en CNF. Chaque clause doit se terminer par un 0 (c'est une convention).

Ainsi la formule $(v_1 \vee v_2 \vee \neg v_3) \wedge (\neg v_1 \vee \neg v_4) \wedge (v_3)$, qui est bien sous forme normale conjonctive (conjonction de clauses) va s'écrire comme :

```
1 2 -3 0
-1 -4 0
3 0
```

On notera que dans le format habituel, le fichier doit commencer par une ligne `"p cnf X Y"` qui indique le nombre max de variables et de clauses dans le fichier. Cette ligne est optionnelle dans `pysat`. On notera également qu'il n'est pas obligatoire que chaque variable apparaisse au moins une fois : on peut avoir des trous. Ainsi, à un renommage près, la formule DIMACS suivante est équivalente à celle du dessus et tout à fait acceptable (tant que l'on reste raisonnable sur le nombre de variables inutilisées) :

```
11 222 -33 0
-11 -41 0
33 0
```

Utilisation d'un solveur SAT "maison" natif en python : pour pouvoir jouer facilement avec les concepts du cours, nous utiliserons PySAT, un solveur efficace (pour un solveur écrit en Python), écrit spécialement pour l'enseignement. Vous pouvez le récupérer sur le dépôt suivant : `https://github.com/lorensipro/pysat`. Pour lancer le solveur, il suffit de se mettre dans le répertoire src et de faire `python pysat.py fichier.cnf` où vous donnez en entrée un fichier au format DIMACS, tel que défini ci-dessus.

Partie 1 : Cerveau contre Monstre

Odeur		*Vent*	**Trou**
Monstre *Odeur*	**Or** *Vent* *Odeur*	**Trou**	*Vent*
Odeur		*Vent*	
Départ	*Vent*	**Trou**	*Vent*

FIGURE 1 – Carte du petit monde du Monstre, d'après « Artificial Intelligence, a Modern Approach »

Explications : Nous allons travailler tout d'abord sur l'exemple vu en cours. Pour rappel, vous partez de la case (0,0) (bas gauche) et vous devez trouver l'or sans rencontrer le monstre ni tomber dans les trous (que nous nommerons par exemple les horribles « trous de l'horreur »). Ce qui va vous aider c'est que l'odeur du monstre se propage aux cases autour du monstre (haut, bas, gauche, droite). Il y a également du vent autour des trous (haut, bas, gauche, droite). Pour s'en sortir, il va falloir réfléchir. Tout est schématisé sur le dessin ci-dessus (tiré du livre « A.I., a Modern Approach »).

1. L'idée de cet exercice est d'utiliser un SAT solveur pour voir si on peut être certain de ses déplacements lors de l'exploration du petit monde. Lancez le programme Python `genMondeMonstre.py` donné avec ce sujet. Ce programme génère les règles du monde, dans une logique propositionnelle avec un nommage des variables lisible. Comment ajouter à ces règles les observations du joueur (le fait qu'il n'y a ni odeur, ni vent en (0,0)) ?
2. Transformez le programme pour qu'il génère maintenant un fichier au format DIMACS. Vous pourrez utiliser le fait que toutes les variables de votre fichier ne sont pas autorisées. Ainsi 100 pourra indiquer qu'il y a un monstre (le 100) en case (0,0). L'encodage d'une case (x,y) sera donc la valeur logique de la variable propositionnelle T * 100 + x *10 + y, où T=1 pour Monstre, 2 pour Odeur, 3 pour Trou, 4 pour Vent et 5 pour Or. La valeur de la variable propositionnelle 324 indiquera, dans cet encodage, s'il y a ou non un Trou en case (2,4).
3. Utilisez votre solveur SAT pour vérifier s'il y a obligatoirement un monstre en case (0,1) (case au dessus du départ). Comment coder cela ?
4. Ajoutez successivement les observations que vous trouverez en vous déplaçant. Cela devrait enrichir au fur et à mesure votre fichier CNF. Attention, avant de vous déplacer sur une case, vérifiez toujours s'il n'y a pas de trou ni de monstre dans la case.
5. Terminez quand vous arrivez sur la case contenant l'or. Les plus aventuriers d'entre vous pourront même automatiser le tout en proposant un navigateur automatique dans la grille !

On notera que, en fait, toutes les contraintes du petit monde sont des contraintes binaires ou unaire. En fait, sur ce genre de problème, on est certain que le solveur SAT va résoudre chaque appel en temps linéaire. C'est une bonne nouvelle, mais cela ne permet pas de montrer tout la puissance de ces solveurs (vous remarquerez que le nombre de conflits donnés par le solveur est toujours très petit).

Partie 2 : Les Sudokus

Pour encoder les Sudokus, il va falloir travailler un peu plus. Nous allons utiliser, ici aussi, le fait que l'on peut avoir des trous dans les variables utilisées. Ainsi, pour encoder en propositionnel le fait que la case (x,y) a la valeur z, nous allons utiliser la variable x*100+y*10+z. C'est possible car toutes les valeurs sont entre 1 et 9. On notera donc que, par exemple, les variable 22 ou 900 ne sont pas utilisées.

Dans ce contexte, la clause :

```
111 112 113 114 115 116 117 118 119 0
```

indique que la case (1,1) dont contenir au moins une valeur parmi [1..9]. Pour vous aider, nous vous donnons un petit bout de code Python permettant de coder que une seule variable parmi un ensemble de variables doit être vraie (contrainte codée par `equals1a`). C'est le fichier `starter-sudoku.py`.

1. Codez une fonction permettant, comme on l'a fait sur le petit monde du monstre, de coder toutes les contraintes respectées par tous les Sudokus. Il faut l'unicité des valeurs sur les colonnes, les lignes, mais aussi les carrés.
2. Prenez un Sudoku Diabolique et ajoutez les indices correspondants pour que PySAT puisse le résoudre.
3. Filtrez les variables du modèle sur uniquement les variables pertinentes du Sudoku et vérifiez que tout est ok. Faites un script permettant de récupérer automatiquement la ligne affichant le modèle pour afficher le sudoku correspondant dans un format classique de Sudoku (et surtout lisible).
4. Comment vérifier que le Sudoku en est vraiment un ? En effet, un Sudoku est un Sudoku s'il n'autorise qu'une unique solution. Vérifiez que vous n'avez pas d'autre solution grâce à un nouvel appel au SAT Solveur.

Bonus

— On aimerait générer un Sudoku. Pour cela, vous devez partir d'une solution aléatoire complète et enlever le maximum d'indices tant que votre Sudoku n'a toujours qu'une seule solution. Imbriquez donc les appels SAT pour que votre programme puisse le faire.